Algorithm Architecture Specification and Software Engineering Proposal

Metadata Field	Project Team Specification
Project Title	Oak Sports Academy Training Registration and Management System Enhancement
Team Name	Oak Sports - Group 5
Section & Year	III-BCSAD
Team Members & Roles	Lead Algorithm Architect: Nathaniel Babalcon Backend / Systems Engineer: Hexel John Ventura, Rommel Villanueva Data / QA Engineer: Jurie Aplaca Systems Analyst / Technical Lead: Aaron Abiera
Repository URL	https://github.com/hex-xel/oak-sports-academy
Submission Date	10/23/2026

Course: ALGOCOM (Algorithms and Complexity) / SOFTENG (Software Engineering)

Executive Summary

This proposal presents the revised algorithmic design of the Oak Sports Academy Training Registration and Management System based on the clarified client workflow. It replaces the previous preference-weighted schedule recommendation algorithm because the revised workflow allows students to select available schedules but does not establish a schedule recommendation feature.

The revised design contains three concrete algorithms:

1. **Classification and Regression Trees (CART)** for explainable student churn-risk classification.
2. **Finite State Machine (FSM) Lifecycle Tracker** for registration, trial

assessment, approval, enrollment, and participation tracking.
3. **Magic Byte File Signature and MIME Validation** for checking uploaded JPEG and PNG documents, supported by bounded image-quality screening.

CART serves as the primary advanced algorithm because decision trees are included among the assignment's supported algorithmic paradigms. The FSM and magic-byte validator satisfy the additional algorithms required by the professor.

The algorithms have separate responsibilities. CART estimates a defined future participation lapse. The FSM records confirmed events and enforces valid lifecycle transitions. The document validator checks uploaded file formats and routes inadequate images for correction or review.

This report covers computational modeling, algorithm mechanics and pseudocode, complexity analysis, decoupled architecture, API contracts, engineering tasks, benchmarking, and academic defense. The presentation deck will be prepared separately.

Task 1 Computational Problem Modeling

1.1 Problem Statement and Domain Context

The revised client workflow requires newly white-belt and colored-belt students to undergo a free-trial assessment before enrollment. After attendance and assessment, authorized academy staff approve or reject the applicant. An accepted student must enroll within one week; otherwise, the account is deleted [1].

After enrollment, the system maintains attendance, package progress, schedules, renewal information, and participant records. These events create three computational needs:

- Maintain consistent participant lifecycle states.
- Identify enrolled participants who may require follow-up because their participation is likely to lapse.
- Check whether uploaded identification documents contain supported image data and meet basic readability requirements.

The FSM addresses lifecycle consistency. CART addresses the professor-required churn-prediction capability. Magic-byte validation addresses secure document collection.

Educational research has investigated predictive classification for identifying students at risk of dropout [6]. However, results from universities cannot establish prediction accuracy for a sports academy. The proposed classifier must therefore be evaluated using representative academy data before operational deployment.

1.2 Primary Computational Problem

The primary problem is **binary classification of future participation lapse**.

Given an enrolled participant's attendance and enrollment information available at snapshot time t, the algorithm estimates whether the participant will have neither an attended session nor an approved renewal during the following 30 days.

This outcome is a proposed operational definition of churn risk. It identifies a participation lapse rather than proving permanent withdrawal.

The 30-day horizon requires academy confirmation before production use.

1.3 Input Parameters

Let the training dataset be:

$$D = \{(x_i, y_i)\}_{i=1}^{n}$$

where:

- n is the number of labeled participant snapshots.
- $x_i \in R^5$ is a five-feature vector.
- $y_i \in \{0,1\}$ is the observed outcome.

Each feature must be calculated using information recorded at or before snapshot time t.

Feature	Definition	Valid Values
x_1	Days since the most recent attended session	Integer ≥ 0
x_2	Consecutive unexcused absences	Integer ≥ 0

x_3	Approved package sessions remaining	Integer ≥ 0
x_4	Attendance rate among the last up to ten eligible scheduled sessions	Real number from 0 to 1
x_5	Days since current enrollment approval	Integer ≥ 0

For x_4, approved medical excusals are excluded from the denominator. If no eligible session exists, the feature is missing and the record is not scored.

Names, addresses, guardian information, uploaded documents, and other unnecessary personal information are excluded from the classifier.

1.4 Outcome Label

For a snapshot at time t:

$$y = \begin{cases} 1, & \text{if no attendance and no approved renewal occur during } (t, t+30 \text{ days}] \\ 0, & \text{otherwise} \end{cases}$$

Only snapshots with a complete, observable 30-day outcome window receive labels.

Snapshots affected by approved medical pauses or incomplete observation are excluded from the initial training cohort.

Future events are used only to establish historical training labels. They must never appear in the predictor values available at time t.

1.5 Boundary Conditions and Constraints

Client workflow constraints

The revised flow establishes the following requirements [1]:

- Required registration information, guardian details for minors, and waiver acceptance must be completed.
- Trial assessment occurs before enrollment.
- Authorized staff determine assessment approval or rejection.

- Accepted applicants must enroll within one week.
- Newly white-belt students are restricted to the applicable group-class and module progression rules.
- Package balances and medical make-ups follow academy approval.
- Unexcused absences may result in session forfeiture.

The classifier consumes the resulting records. It does not determine assessment approval, module eligibility, session balances, payment status, or account access.

Proposed model constraints

Initial engineering settings are:

- Number of features: $d = 5$.
- Maximum tree depth: $H = 8$.
- Minimum examples per leaf: $L = 5$.
- Classification threshold: $\tau = 0.5$.
- Minimum accepted impurity improvement: $\varepsilon = 10^{-12}$.

These are proposed settings, not confirmed client requirements.

Training readiness requires:

- Both outcome classes.
- Complete feature values and mature labels.
- Participant-disjoint training, validation, and test partitions.
- Chronological separation between those partitions.

A proposed initial feasibility gate is at least 100 mature snapshots from 50 participants, including at least 20 examples of each class overall and both classes in each partition. These counts do not guarantee statistical adequacy.

1.6 Objective Function

CART constructs a binary classification tree through greedy recursive partitioning. At each node, it selects the valid feature threshold that produces the greatest reduction in weighted Gini impurity [5].

For a node containing m examples, of which c have label 1:

$$p = \frac{c}{m}$$

$$G(m, c) = 1 - p^2 - (1 - p)^2$$

For a proposed split into left and right subsets:

$$Gain = G(m, c) - \frac{m_L}{m} G(m_L, c_L) - \frac{m_R}{m} G(m_R, c_R)$$

The algorithm selects the largest positive gain subject to the depth and minimum-leaf constraints.

This is a local greedy objective. It does not guarantee the globally optimal complete decision tree.

1.7 Output Certificates

Algorithm	Successful Output	Controlled Alternative
CART training	Versioned tree, feature schema, parameters, node counts, and evaluation metadata	INSUFFICIENT_DATA
CART prediction	Risk label, leaf frequency, support count, and decision path	NOT_SCORED
Lifecycle FSM	Previous state, accepted event, next state, version, and applicable deadline	INVALID_TRANSITION, GUARD_FAILED, or STALE_EVENT
Image validation	Verified format, dimensions, actual byte count, checksums, and opaque storage key	Signature, MIME, size, structural, or quality-review result

1.8 Cold Start Behavior

When a validated model is unavailable, the system returns:

- status: NOT_SCORED
- mode: RULE_BASELINE
- probability: null

A proposed temporary rule flags FOLLOW_UP_REQUIRED after at least three consecutive unexcused absences. Otherwise, it returns NO_RULE_TRIGGER.

This rule requires academy confirmation. It is not a learned prediction, and the absence of a flag does not establish low risk.

Synthetic data may demonstrate algorithm mechanics and runtime behavior. It cannot establish real academy prediction accuracy.

<!-- AI Assistance: ChatGPT assisted with the problem formulation, feature definitions, proposed outcome label, and mathematical model.-->

Task 2 Algorithmic Paradigm and Logic

2.1 Primary Algorithm Specification

Schema Field	Engineering Specification
Algorithm Name	CART Binary Classification Tree using Gini Impurity
Design Paradigm	Supervised Machine Learning with Greedy Recursive Partitioning
Foundational Reference	Raschka, Liu, and Mirjalili, 2022 [5]
Baseline Alternatives	Majority-class prediction and the documented absence rule
Computational Justification	Sorted feature values and prefix counts avoid recounting all labels for every candidate threshold
Main Invariant	Each node owns a row subset; its children form a disjoint partition of that subset

Termination Conditions	Maximum depth, pure labels, insufficient examples, or no meaningful impurity improvement

Split rules

For feature j and threshold a:

- Left child: $x_j \leq a$.
- Right child: $x_j > a$.

Only thresholds between distinct observed values are considered. Each child must contain at least L examples.

The threshold is the lower observed value at a valid boundary. This avoids calculating a potentially overflowing midpoint.

For reproducibility:

1. Process features in their documented order.
2. Process candidate thresholds in ascending order.
3. Preserve the first candidate when gains differ by no more than ε.

2.2 CART Training Pseudocode

The training algorithm receives immutable feature and label arrays. It maintains one global row-index array and partition index ranges in place.

```
FUNCTION TRAIN_CART(X, y, H = 8, L = 5, epsilon = 1e-12):

  VALIDATE:
    X is a nonempty rectangular numeric matrix
    every feature is finite and satisfies its defined range
    y contains one binary label for every row
    H >= 0
    L >= 1
    number of rows >= L

  I <- [0, 1, ..., n - 1]

  root <- BUILD(0, n, 0)

  RETURN root and training metadata
```

```
FUNCTION BUILD(lo, hi, depth):

   m <- hi - lo
   c <- number of positive labels among I[lo : hi]

   leaf <- {
      positiveCount: c,
      sampleCount: m,
      frequency: c / m
   }

   IF depth = H
     OR c = 0
     OR c = m
     OR m < 2 * L:
      RETURN leaf

   parentImpurity <- GINI(m, c)
   bestGain <- 0
   bestSplit <- NONE

   FOR feature j FROM 0 TO d - 1:

      A <- temporary records containing:
           feature value, label, and row index
           for every row in I[lo : hi]

      SORT A by feature value, then row index

      leftPositive <- 0

      FOR k FROM 1 TO m - 1:

         leftPositive <- leftPositive + A[k - 1].label

         IF k < L OR m - k < L:
            CONTINUE

         IF A[k - 1].value = A[k].value:
            CONTINUE

         rightPositive <- c - leftPositive
```

```
        gain <- parentImpurity
              - (k / m) * GINI(k, leftPositive)
              - ((m - k) / m)
                * GINI(m - k, rightPositive)

        IF gain > bestGain + epsilon:
           bestGain <- gain
           bestSplit <- (j, A[k - 1].value)

     DISCARD A before recursion

  IF bestSplit = NONE:
     RETURN leaf

  (j, threshold) <- bestSplit

  mid <- PARTITION_INDICES(
     I, lo, hi, j, threshold
  )

  leftChild <- BUILD(lo, mid, depth + 1)
  rightChild <- BUILD(mid, hi, depth + 1)

  RETURN {
     feature: j,
     threshold: threshold,
     left: leftChild,
     right: rightChild,
     positiveCount: c,
     sampleCount: m
  }


FUNCTION PARTITION_INDICES(I, lo, hi, j, threshold):

  left <- lo
  right <- hi - 1

  WHILE left <= right:

     IF X[I[left]][j] <= threshold:
        left <- left + 1
```

```
    ELSE:
      SWAP I[left] and I[right]
      right <- right - 1

  RETURN left


FUNCTION GINI(m, c):

  REQUIRE m > 0

  p <- c / m

  RETURN 1 - p * p - (1 - p) * (1 - p)
```

Training and deployment are separate steps. A tree constructed successfully from demonstration data must not become an operational predictor unless the data-readiness and held-out evaluation requirements are satisfied.

2.3 CART Prediction Pseudocode

```
FUNCTION PREDICT(model, x, threshold = 0.5):

  IF model is unavailable:
    RETURN NOT_SCORED with reason NO_MODEL

  IF x has missing, nonfinite, or out-of-range features:
    RETURN NOT_SCORED with reason INVALID_FEATURES

  IF feature schema does not match model:
    RETURN NOT_SCORED with reason SCHEMA_MISMATCH

  node <- model.root
  path <- empty list

  WHILE node is not a leaf:

    j <- node.feature
    a <- node.threshold

    IF x[j] <= a:

      APPEND (j, a, LEFT) to path
```

```
        node <- node.left

      ELSE:

        APPEND (j, a, RIGHT) to path
        node <- node.right

  frequency <- node.positiveCount / node.sampleCount

  IF frequency >= threshold:
     label <- HIGH_RISK
  ELSE:
     label <- LOWER_RISK

  RETURN {
     status: SCORED,
     modelVersion: model.version,
     riskLabel: label,
     leafFrequency: frequency,
     leafSupport: node.sampleCount,
     decisionPath: path
  }
```

The leaf frequency is the proportion of positive training examples reaching that leaf. It is not a guarantee of the participant's future outcome or a calibrated probability.

2.4 Worked CART Example

Consider six synthetic examples using consecutive unexcused absences:

Example	Consecutive Unexcused Absences	Observed Lapse Label
A	0	0
B	0	0
C	1	0
D	3	1

E	4	1
F	5	1

For illustration, use $L = 1$.

The parent node contains three examples from each class:

$$G(parent) = 1 - (3/6)^2 - (3/6)^2 = 0.5$$

A split at:

$$x_2 \leq 1$$

produces:

- Left: labels *0,0,0*.
- Right: labels *1,1,1*.

Both children have impurity 0:

$$Gain = 0.5 - 0 - 0 = 0.5$$

A new example with four consecutive unexcused absences follows the right branch. Its leaf frequency is $3/3 = 1$.

This example demonstrates split selection. It does not establish operational accuracy.

2.5 Correctness and Termination

Every accepted split creates two smaller, nonempty subsets containing at least L examples. Recursion also increases depth toward H. Training therefore terminates.

During a feature scan, the prefix count equals the positive-label count in the proposed left child. Subtracting it from the parent count gives the right-child count.

The algorithm evaluates every legal distinct-value boundary and selects the greatest measured local gain, subject to numerical tolerance and tie rules.

Prediction follows one child at each internal node and terminates at a leaf within the tree's realized depth.

<!-- AI Assistance: ChatGPT assisted with CART pseudocode, the worked example, and correctness explanations. The team must review the implementation logic and mathematical calculations. -->

2.6 Supporting Algorithm Lifecycle FSM

Purpose

The FSM records confirmed participant progress and rejects invalid lifecycle transitions. It supports trial conversion monitoring while remaining separate from the predictive classifier.

Finite-state modeling represents behavior through a finite set of states and event-driven transitions [12].

Define:

$$\delta(s, e, g) = s'$$

where:

- s is the current lifecycle state.
- e is an authenticated event.
- g represents required business guards.
- s' is the next state.

Unlisted state/event combinations are rejected.

Proposed lifecycle states

State	Meaning
REGISTERED	Registration requirements are complete
TRIAL_SCHEDULED	Staff assigned the assessment schedule

ASSESSMENT_PENDING	Trial attendance is recorded; the assessment decision is pending
APPROVED_AWAITING_ENROLLMENT	Assessment approved; the one-week enrollment window is active
REJECTED	Staff rejected the assessment
ENROLLED	Enrollment is approved and confirmed
INACTIVE	Participation inactivity is confirmed under an approved policy
WITHDRAWN	Formal withdrawal is confirmed
DELETION_DUE	The accepted applicant missed the enrollment deadline and account removal must execute

Transition table

Current State	Trigger Event	Required Guard	Next State
REGISTERED	TRIAL_SCHEDULED	Authorized staff; required verification, guardian information, and waiver complete	TRIAL_SCHEDULED

TRIAL_SCHEDULED	ASSESSMENT_ATTENDED	Authorized attendance recorded for the scheduled trial	ASSESSMENT_PENDING
ASSESSMENT_PENDING	ASSESSMENT_APPROVED	Authorized staff assessment approval	APPROVED_AWAITING_ENROLLMENT
ASSESSMENT_PENDING	ASSESSMENT_REJECTED	Authorized staff assessment rejection	REJECTED
APPROVED_AWAITING_ENROLLMENT	ENROLLMENT_CONFIRMED	Before deadline; eligible enrollment approved with required fee confirmation	ENROLLED
APPROVED_AWAITING_ENROLLMENT	DEADLINE_REACHED	Deadline reached with no timely confirmed enrollment	DELETION_DUE
ENROLLED	INACTIVITY_CONFIRMED	Approved inactivity policy or staff determination; medical pauses	INACTIVE

		excluded	
INACTIVE	RENEWAL_CONFIRMED	Eligible renewal approved	ENROLLED
ENROLLED	RENEWAL_CONFIRMED	Renewal approved; balance changes handled separately	ENROLLED
Any nonterminal state	WITHDRAWAL_CONFIRMED	Authorized confirmation of participant or guardian request	WITHDRAWN

REJECTED, WITHDRAWN, and DELETION_DUE are terminal in this initial proposal. Reapplication, trial rescheduling, and trial no-shows require additional academy rules.

FSM pseudocode

```
FUNCTION APPLY_EVENT(participantId, event, now):

  REQUIRE authenticated actor
  REQUIRE actor is authorized for participant and event type

  BEGIN TRANSACTION

  LOCK participant record
  LOAD current state and version
```

```
IF event.id was already committed:
   LOAD stored result
   END TRANSACTION
   RETURN stored result

IF event.expectedVersion != current.version:
   ROLLBACK
   RETURN STALE_EVENT

rule <- transitionTable[current.state, event.type]

IF rule does not exist:
   ROLLBACK
   RETURN INVALID_TRANSITION

IF rule.guard(current, event, now) is false:
   ROLLBACK
   RETURN GUARD_FAILED

nextState <- rule.targetState

IF nextState = APPROVED_AWAITING_ENROLLMENT:
   acceptedAt <- authoritative approval timestamp
   deadline <- acceptedAt + 7 days

UPDATE lifecycle state and applicable timestamps
INCREMENT version

STORE event result
APPEND audit entry in the same transaction

IF nextState = DELETION_DUE:
   CREATE account-removal instruction atomically

COMMIT

RETURN previous state, next state, event ID,
     new version, and applicable deadline
```

Deadline handling

The client requires deletion if an accepted student does not enroll within one week [1].

Proposed implementation semantics are:

- Store timestamps as UTC instants.
- Display deadlines in Asia/Manila time.
- Interpret one week as seven consecutive 24-hour periods.
- Accept enrollment strictly before the deadline.
- At or after the deadline, trigger account removal if no timely enrollment exists.

The exact cutoff interpretation requires academy confirmation.

Enrollment confirmation and deadline processing must use the same participant lock or equivalent concurrency control. This prevents both operations from succeeding against stale information.

DELETION_DUE indicates required cleanup; it does not mean deletion has already succeeded. The removal service must verify completion. Deletion scope and permissible reporting or audit retention require owner approval.

Worked lifecycle trace

An applicant follows:

REGISTERED
→ TRIAL_SCHEDULED
→ ASSESSMENT_PENDING
→ APPROVED_AWAITING_ENROLLMENT
→ ENROLLED

If approval occurs on November 4, 2026, at 10:00 Asia/Manila time, the proposed deadline is November 11 at 10:00.

Enrollment confirmed on November 10 produces ENROLLED.

If no timely enrollment exists when the deadline is reached:

APPROVED_AWAITING_ENROLLMENT
→ DELETION_DUE
→ Account removal service

Replaying the same event ID returns its stored result without another mutation.

Trial conversion metric

For a mature acceptance cohort:

$$ConversionRate_7 = \frac{Acceptedtrialparticipantsenrolledwithinsevendays}{Allacceptedtrialparticipantsinthecohort} \times 100$$

Count each participant once. Rejected assessments are excluded.

If the denominator is zero or the cohort's enrollment windows have not all elapsed, return NOT_AVAILABLE.

<!-- AI Assistance: ChatGPT assisted with the FSM states,
transition table, event-processing pseudocode, and conversion metric.
The proposed deadline and retention details require owner review. -->

2.7 Supporting Algorithm Magic Byte and MIME Validation

Purpose

The validator checks whether an uploaded document begins with a supported image signature and whether that detected format agrees with the declared MIME type and filename extension.

It prevents accepting a plain script merely because it was renamed with an image extension.

A matching signature is only a preliminary format check. It cannot prove that the entire file is safe, structurally valid, or readable.

Supported signatures

Format	Signature	MIME Type	Extensions
PNG	89 50 4E 47 0D 0A 1A 0A	image/png	.png
JPEG candidate	FF D8 FF	image/jpeg	.jpg, .jpeg

The PNG signature is defined by the PNG specification [7].

The JPEG prefix requires subsequent structural validation. Valid JPEGs must not be restricted to only JFIF or Exif application markers.

Proposed screening limits

- Minimum actual size: 5 KiB.
- Maximum actual size: 5 MiB.
- Minimum dimensions: 400 × 400 pixels.
- Maximum decoded image size: 20 megapixels.
- Supported formats: static JPEG and PNG.
- Unsupported document formats, including SVG, are rejected.

Here, 1 KiB means 1,024 bytes and 1 MiB means 1,048,576 bytes.

The initial size and dimension limits are based on the existing project specification [3]. Their operational suitability requires review.

Validation pseudocode

```
FUNCTION VALIDATE_IMAGE(
  stream, declaredMime, extension, categoryRules
):

  head <- READ up to 8 initial bytes

  IF fewer than 8 bytes are available:
    RETURN INVALID_SIGNATURE

  IF head matches PNG signature:
    detectedMime <- image/png
    allowedExtensions <- {.png}

  ELSE IF first 3 bytes match FF D8 FF:
    detectedMime <- image/jpeg
    allowedExtensions <- {.jpg, .jpeg}

  ELSE:
    RETURN INVALID_SIGNATURE

  IF normalized declaredMime != detectedMime:
    RETURN MIME_MISMATCH

  IF lowercase extension is not allowed:
    RETURN MIME_MISMATCH

  REPLAY head into a bounded quarantine stream

  actualBytes <- 0
```

```
INITIALIZE incremental source checksum

FOR each received chunk:

    actualBytes <- actualBytes + chunk.length

    IF actualBytes > MAX_BYTES:
        ABORT
        RETURN SIZE_LIMIT

    UPDATE source checksum
    WRITE chunk to quarantine

IF actualBytes < MIN_BYTES:
    RETURN SIZE_LIMIT

PARSE image using bounded marker or chunk lengths

IF malformed, truncated, animated, or unsupported:
    RETURN MALFORMED_IMAGE

width, height <- validated dimensions

IF width < MIN_WIDTH OR height < MIN_HEIGHT:
    RETURN IMAGE_DIMENSIONS_TOO_SMALL

IF width * height > MAX_PIXELS:
    RETURN PIXEL_LIMIT

DECODE with explicit memory and time limits

IF decoding fails:
    RETURN MALFORMED_IMAGE

contrast <- grayscale variance
sharpness <- variance of Laplacian

IF calibrated quality thresholds are not satisfied:
    RETURN QUALITY_REVIEW with recapture guidance

RE-ENCODE as an allowed image format
REMOVE unnecessary metadata and trailing data

GENERATE random server-side filename
```

STORE through an authorized private-storage adapter

CALCULATE checksum of stored output
RECORD source and stored checksums separately

RETURN verified MIME, dimensions, byte count,
stored checksum, and opaque storage key

The server counts bytes actually received. It does not rely on an untrusted Content-Length declaration.

Image-quality heuristics

For a grayscale image I, contrast is:

$$Var(I) = mean(I^2) - mean(I)^2$$

For each interior pixel:

$$L(x,y) = I(x-1,y) + I(x+1,y) + I(x,y-1) + I(x,y+1) - 4I(x,y)$$

Sharpness is estimated using:

$$Var(L) = mean(L^2) - mean(L)^2$$

The implementation accumulates sums and squared sums over the relevant pixels.

Thresholds must be calibrated using consented examples for each document category. Until calibration is complete, questionable images should receive staff review rather than an unsupported automatic rejection.

These heuristics do not establish document authenticity or guarantee text legibility. File handling must also follow authorization, resource-limit, and private-storage controls [8].

Worked validation examples

Upload	Detected Content	Result

valid-id.png, declared image/png, valid PNG structure	PNG	Continue to size, dimensions, decode, and quality checks
script.jpg, declared image/jpeg, begins with script text	Unsupported signature	INVALID_SIGNATURE
photo.png, declared image/png, contains JPEG bytes	JPEG candidate	MIME_MISMATCH
Valid PNG prefix followed by truncated data	Incomplete PNG	MALFORMED_IMAGE
Valid image with inadequate dimensions	Supported image	IMAGE_DIMENSIONS_TOO_SMALL

<!-- AI Assistance: ChatGPT assisted with signature-validation pseudocode and proposed image-screening heuristics. Thresholds must be reviewed and calibrated before automatic quality rejection. -->

Task 3 Theoretical Complexity and Scalability

3.1 CART Training Time Complexity

Let:

- n: training examples.
- d: features.
- H: maximum permitted depth.
- h: realized depth.
- T: tree nodes.

At a node containing m examples:

1. Counting labels costs $O(m)$.

2. Sorting one feature costs $O(m \log m)$.
3. Scanning its candidate boundaries costs $O(m)$.
4. Repeating across d features costs $O(dm \log m)$.
5. Partitioning row indices costs $O(m)$.

The recursive cost is therefore:

$$C(m) = C(m_L) + C(m_R) + O(dm \log m)$$

At a particular depth, node subsets are disjoint and contain at most n examples in total. Their combined work is bounded by:

$$O(dn \log n)$$

Across h levels:

$$O(dnh \log n)$$

With maximum depth H, including initial validation:

$$O(nd + dnH \log n)$$

For fixed $d = 5$ and $H = 8$, the upper bound simplifies to:

$$O(n \log n)$$

This implementation sorts again at each node. Its unrestricted balanced-tree bound is:

$$O(dn \log^2 n)$$

A conservative unrestricted, highly skewed bound is:

$$O(dn^2 \log n)$$

These are implementation-specific derivations using standard asymptotic analysis [4]. They must not be replaced with the complexity of a different implementation that reuses presorted feature orders.

Best and average cases

A pure root or an immediate size-based stop requires only root counting after validation. End-to-end processing is $O(nd)$.

An unconditional average-case bound is not asserted because no training-data distribution is assumed. The bounded-depth upper bound remains applicable.

3.2 CART Prediction Complexity

Prediction follows one root-to-leaf path.

- Best case: $O(1)$.
- Worst case for the constructed tree: $O(h)$.
- With the proposed depth cap: at most $O(H)$.
- Balanced unrestricted tree: $O(log\ n)$.
- Highly skewed unrestricted tree: up to $O(n)$.

3.3 CART Space Complexity

Excluding the read-only $O(nd)$ input:

- Global row-index array: $O(n)$.
- Reusable feature-sorting scratch space: $O(n)$.
- Recursion stack: $O(h)$.
- Output tree: $O(T)$.

Total auxiliary space is:

$$O(n + h + T)$$

This bound assumes scratch buffers are released before recursion and the global index array is partitioned in place.

The number of nodes is bounded by:

\[T\le \min\left(2\left\lfloor\frac{n}{L}\right\rfloor-1,\;2^{H+1}-1\right) \]

Prediction requires $O(1)$ traversal space, or $O(h)$ when returning the explanation path.

3.4 FSM Complexity

A fixed transition table provides:

- Dispatch time: $O(1)$.
- Transient dispatch space: $O(1)$.

These bounds assume required guard facts are already available.

They exclude database queries, locks, audit persistence, notifications, deadline discovery, and account removal.

If E lifecycle events are retained, historical storage is $O(E)$. A full scan of N participants to discover expired deadlines costs $O(N)$.

3.5 Image Validation Complexity

Let:

- k: inspected signature bytes, at most eight.
- B: received file bytes.
- P: decoded pixels.
- C: fixed streaming-buffer capacity.

Signature checking costs:

$$O(k) = O(1)$$

The full pipeline includes streaming, hashing, structural parsing, decoding, quality scans, and re-encoding:

$$O(B + P)$$

A bounded implementation uses:

$$O(C + P)$$

auxiliary memory.

JPEG dimension markers may occur beyond the first 32 bytes. Therefore, the complete image pipeline must not be described as constant time.

3.6 Big Data Dimensions

Dimension	Engineering Parameter	Scalability Impact and Mitigation
Volume	Synthetic training sizes of 100, 1,000, and 5,000 rows	Train offline; cap depth; bound memory and image dimensions
Velocity	Proposed stress sweep of 1, 10, and 25 requests per second	Measure inference latency separately from training, storage, and database access
Variety	Numeric snapshots, lifecycle events, timestamps, and image bytes	Validate versioned schemas and normalize authoritative timestamps and formats

These values are proposed evaluation workloads, not measured academy production traffic.

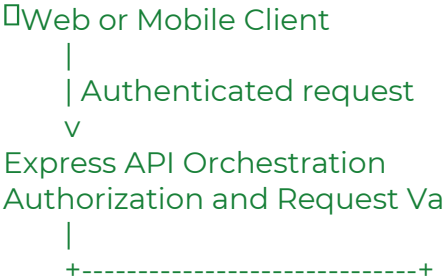
<!-- AI Assistance: ChatGPT assisted with implementation-specific complexity derivations and scalability assumptions. The team must verify that the eventual implementation matches these assumptions. -->

Task 4 Decoupled Architecture and Data Contract

4.1 Architecture

The algorithms should operate independently of database models, HTTP request objects, and UI components.

The existing Express backend remains responsible for authorization, authoritative data retrieval, validation, and persistence.



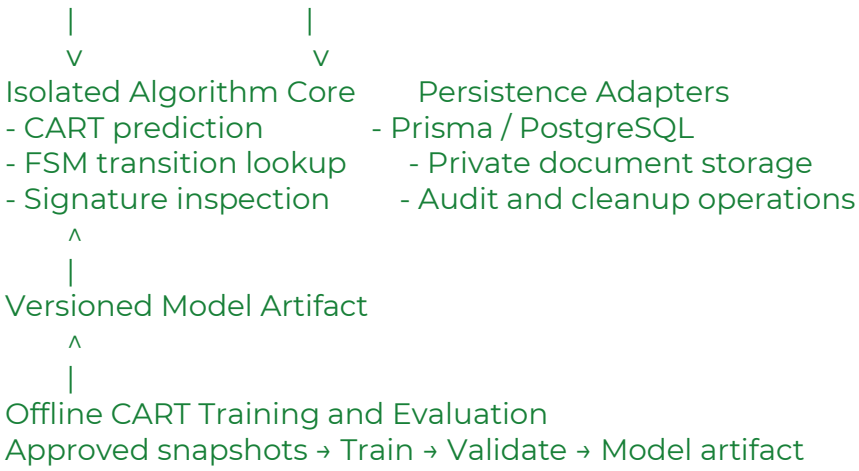


Figure 1. Decoupled algorithm architecture. The core receives validated data-transfer objects and returns certificates. Persistence remains outside mathematical computation.

Training runs outside the web request path. Node.js worker threads may be considered for CPU-intensive execution if measurements justify them [11]. This proposal does not require a new external service or framework.

4.2 Proposed Module Boundaries

Module	Responsibility
apps/api/src/modules/analytics/algorithm/	CART construction, prediction, and model serialization
apps/api/src/modules/participants/lifecycle/	Transition lookup and lifecycle guard evaluation
apps/api/src/modules/documents/validation/	Signature validation and bounded image-screening coordination
API orchestration	Authorization, authoritative snapshots, transactions, persistence, and safe errors

These are proposed implementation paths, not claims that the modules already exist.

4.3 Prediction Request Contract

Proposed endpoint:

POST /api/v1/algorithm/solve

This endpoint is restricted to authorized staff.

```
{
  "requestId": "req-demo-001",
  "operation": "CHURN_PREDICT",
  "participantId": "participant-demo-001",
  "asOf": "2026-11-04T02:00:00Z"
}
```

Requirements:

- All four fields are required.
- requestId and participantId contain 1–64 ASCII letters, digits, or hyphens.
- operation must equal CHURN_PREDICT.
- asOf must be a valid UTC date-time and cannot be in the future.
- The proposed endpoint rejects additional fields.
- The server retrieves features and selects the active model.

The client does not submit trusted lifecycle facts, feature values, or model filenames.

4.4 Internal Engine DTO

```
{
  "modelVersion": "cart-demo-v1",
  "features": {
    "daysSinceAttendance": 12,
    "unexcusedAbsenceStreak": 3,
    "sessionsRemaining": 2,
    "attendanceRateLast10": 0.4,
    "daysSinceEnrollment": 45
  }
}
```

The backend constructs this object after authorization and snapshot validation.

4.5 Response Certificate

```
{
 "requestId": "req-demo-001",
 "status": "SCORED",
 "mode": "CART",
 "modelVersion": "cart-demo-v1",
 "riskLabel": "HIGH_RISK",
 "leafFrequency": 0.8,
 "leafSupport": 10,
 "decisionPath": [
  {
   "feature": "unexcusedAbsenceStreak",
   "threshold": 2,
   "branch": "RIGHT"
  }
 ],
 "executionMetrics": {
  "nodesVisited": 2,
  "runtimeMs": 0.0
 }
}
```


The values above are illustrative. Actual runtime values must be measured.

An unavailable model produces a NOT_SCORED response with rule-baseline mode and no predicted probability.

The API schemas should be documented using OpenAPI 3.1 [9]. HTTP responses follow standard HTTP semantics [10]:

HTTP Status	Meaning
200	Scored or controlled not-scored result
400	Malformed request
401	Authentication required
403	Insufficient permission

404	Requested authorized record unavailable
409	Stale lifecycle event
413	Upload exceeds size limit
415	Unsupported media type
500	Unexpected failure with a generic error identifier

4.6 Lifecycle and Upload Contracts

Lifecycle event requests contain:

- Event ID.
- Participant ID.
- Expected record version.
- Event type.
- Required evidence identifiers.

The server derives timestamps and guard facts from authoritative records.

Document uploads use multipart/form-data containing the document category and file bytes. Files are not embedded as base64 in the prediction request.

Errors must provide safe reason codes and recovery guidance without exposing participant data, stack traces, or decoder internals [8].

<!-- AI Assistance: ChatGPT assisted with the architecture diagram, module boundaries, and API examples. These interfaces are proposals for team review and implementation. -->

Task 5 Agile Backlog and Empirical Validation

5.1 User Stories

User Story 1

As academy staff, I want an explainable churn-risk result from authorized attendance and enrollment snapshots so that I can prioritize participant follow-up without changing enrollment eligibility.

User Story 2

As an academy administrator, I want assessment and enrollment events validated by a lifecycle FSM and uploaded IDs checked against their actual image format so that participant records follow the revised workflow and inadequate uploads can be corrected.

5.2 Engineering Implementation Tickets

DEV-ALGO-101 CART Classification

Proposed module: apps/api/src/modules/analytics/algorithm/

Implementation steps:

1. Define the five-feature schema and mature outcome label.
2. Construct participant-disjoint chronological partitions.
3. Implement Gini calculation and prefix-count split evaluation.
4. Implement in-place index partitioning.
5. Implement bounded recursive tree construction.
6. Implement prediction and explanation paths.
7. Serialize versioned model artifacts.
8. Compare held-out results against documented baselines.

Acceptance criteria:

- Invalid and incomplete inputs produce controlled results.
- Pure and constant-feature datasets terminate.
- Repeated feature values do not create invalid boundaries.
- Every child leaf satisfies the configured minimum size.
- Tree depth does not exceed eight.
- Small fixtures match hand-calculated splits.
- Model absence produces NOT_SCORED.
- Predictions remain advisory.

DEV-ALGO-102 Lifecycle FSM

Proposed module: apps/api/src/modules/participants/lifecycle/

Implementation steps:

1. Obtain approval for specification reconciliation.
2. Implement the transition table and guard evaluation.
3. Add event idempotency and version checks.
4. Persist transitions and audit records atomically.
5. Implement deadline discovery.
6. Implement account removal after deletion scope is approved.

Acceptance criteria:

- Enrollment before assessment approval is rejected.
- Assessment rejection cannot enter the accepted-enrollment path.
- Timely enrollment and timeout boundaries behave consistently.
- Concurrent enrollment and timeout processing cannot both succeed.
- Event replay causes no duplicate mutation.
- Cleanup completion is verified separately from DELETION_DUE.

DEV-ALGO-103 Document Validation

Proposed module: apps/api/src/modules/documents/validation/

Implementation steps:

1. Implement PNG and JPEG prefix inspection.
2. Cross-check MIME type and extension.
3. Enforce actual received-byte limits.
4. Validate structure and dimensions.
5. Decode within resource limits.
6. Calculate quality indicators.
7. Re-encode and store through an authorized private-storage adapter.
8. Return safe correction or retry guidance.

Acceptance criteria:

- Renamed scripts are rejected.
- MIME and extension mismatches are rejected.
- Truncated images fail safely.
- Valid JPEGs are not restricted to only JFIF or Exif markers.
- Excessive dimensions and byte counts are rejected.
- Storage failure permits a safe retry.
- Quality rejection remains disabled until thresholds are calibrated.

5.3 Correctness Validation

Use deterministic fixtures covering:

- Empty data.
- Pure labels.
- Constant features.
- Repeated values.
- Equal split gains.
- Minimum leaf boundaries.
- Maximum depth.
- Highly uneven partitions.
- Missing or invalid features.
- Invalid lifecycle transitions.
- Duplicate and stale events.
- Exact enrollment-deadline boundaries.
- Valid, mismatched, truncated, and oversized images.

Small classification examples should be checked manually before larger benchmarks.

5.4 Predictive Evaluation

For real academy data:

1. Separate participants into training, validation, and test groups.
2. Order snapshot windows chronologically.
3. Require training outcome windows to finish before validation snapshots begin.
4. Apply the same separation before the test period.
5. Never place one participant in multiple partitions.
6. Tune settings using validation data only.
7. Report final test results without repeatedly tuning against them.

Report:

- Confusion matrix.
- Positive-class precision.
- Recall.
- F1 score.
- Precision–recall area under the curve.
- Class prevalence.
- Sample counts and uncertainty.
- Comparison with majority-class and rule baselines.

Accuracy alone is insufficient when lapse outcomes are uncommon [5, 6].

If the classifier fails to improve a pre-agreed follow-up metric, retain the rule baseline and continue collecting suitable data.

5.5 Proposed Benchmark Assertions

Component	Workload	Initial Assertion
CART training	5,000 rows, five features, depth eight, minimum leaf five	Median core runtime below two seconds
CART prediction	10,000 predictions with model preloaded	Median batch runtime below 250 ms
FSM dispatch	10,000 pure transition lookups	Median batch runtime below 250 ms
Signature validation	10,000 eight-byte checks	Median batch runtime below 250 ms

These are proposed targets, not measured results.

Training measurements include sorting, split evaluation, partitioning, and model construction. They exclude dataset generation and file I/O.

Inference measurements include explanation construction but exclude network and database access.

FSM and signature benchmarks exclude persistence, decoding, storage, and account removal. Those costs require separate measurements.

Benchmark procedure

Use deterministic workloads of 100, 1,000, and 5,000 training examples. Perform one warm-up followed by at least ten measured repetitions.

Record:

- Median and p95 runtime.
- Peak memory.
- Runtime and operating-system versions.

- Processor and RAM.
- Dataset seed.
- Feature count and depth.
- Exact measurement boundaries.
- Relevant garbage-collection settings.

The project owner runs application test suites and benchmarks under the repository execution boundary. No empirical benchmark result is claimed in this proposal.

<!-- AI Assistance: ChatGPT assisted with user stories,
implementation tickets, correctness cases, and the benchmarking plan.
All future results must come from the team's own measurements. -->

Task 6 Academic Review and Defense

6.1 Why Must Heavy Algorithmic Processing Be Decoupled From Database ORMs and Web Request Threads

The algorithmic core should receive plain validated data and return a certificate. Database access, authorization, and transactions remain orchestration responsibilities.

This separation makes algorithm behavior reproducible and prevents mathematical logic from depending on database schemas or UI components.

Training and image decoding may occupy substantial CPU time. Training therefore runs offline, while decoding requires resource limits. Worker execution should be considered only when measurements justify it [8, 11].

6.2 If the Algorithm Has Exponential Worst-Case Complexity What Maintains Production Viability

The specified CART construction uses polynomial greedy split search with a bounded depth:

$$O(nd + dnH\ log\ n)$$

It does not enumerate every possible decision tree.

Maximum depth, minimum leaf size, and positive-gain stopping rules bound computation and reduce overfitting. A future requirement for globally optimal tree construction would require a different formulation and analysis [4, 5].

6.3 What Is the Difference Between Exact and Heuristic Algorithms

An exact optimizer guarantees an optimum for its stated model.

Greedy CART selects locally beneficial splits and can miss a better complete tree. Prediction follows the resulting tree deterministically, but the participant's future behavior remains uncertain.

FSM and signature checks apply explicit rules. They do not provide predictive accuracy or prove universal file safety [5, 7, 8].

6.4 How Does Empirical Profiling Differ From Big O Analysis

Big-O describes how resource use grows under defined input sizes and cost assumptions.

Benchmarking measures a particular implementation on a named machine, runtime, and workload.

A fast experiment cannot prove an asymptotic bound, and a polynomial bound cannot establish an actual latency target. Both forms of evidence are necessary and must state their boundaries [4].

6.5 How Will the Project Handle Cold Start and Sparse Data

The FSM and signature checker operate without learned history.

CART requires mature labeled snapshots and held-out evaluation. Until these exist, the system returns NOT_SCORED and uses the separately labeled rule baseline without a probability.

Missing predictors are not invented. Synthetic data demonstrates mechanics only. Participant-disjoint temporal partitions and minimum leaf sizes help control leakage and sparse leaves, but representative academy data remains necessary [5, 6].

6.6 How Are Edge Cases Handled Without Crashing the Host Service

The implementation validates feature shape, ranges, finite values, and model versions before prediction.

Training stops at pure or unsplittable nodes. The FSM rejects invalid pairs and stale versions and makes event retries idempotent.

Upload processing enforces byte, pixel, memory, and time limits. Malformed images produce safe errors.

Classifier results cannot grant access, approve assessments, or bypass enrollment rules. Correctness, transaction races, and resource-limit behavior require separate owner-run verification [8–10].

<!-- AI Assistance: ChatGPT assisted with the academic defense explanations. The team must understand and verify the arguments before submission and oral defense. -->

References

T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022. Publisher page.

[5] S. Raschka, Y. Liu, and V. Mirjalili, *Machine Learning with PyTorch and Scikit-Learn*. Birmingham, UK: Packt, 2022, particularly Chapters 3 and 6. ISBN 9781801819312. Author companion repository.

[6] A. Gonzalez-Nucamendi, J. Noguez, L. Neri, V. Robledo-Rella, and R. M. G. García-Castelán, "Predictive analytics study to determine undergraduate students at risk of dropout," *Frontiers in Education*, vol. 8, article 1244686, 2023. DOI: 10.3389/feduc.2023.1244686.

[7] World Wide Web Consortium, *Portable Network Graphics PNG Specification Third Edition*, W3C Recommendation, June 24, 2025. Specification.

[8] OWASP Foundation, *Application Security Verification Standard*, version 5.0.0, May 30, 2025. Standard website.

[9] OpenAPI Initiative, *OpenAPI Specification*, version 3.1.0, February 15, 2021. Specification.

[10] R. Fielding, M. Nottingham, and J. Reschke, Eds., *HTTP Semantics*, RFC 9110, IETF, June 2022. RFC 9110.

[11] Node.js Contributors, "Worker threads," Node.js v26.13.0 documentation, consulted November 4, 2026. Documentation.

[12] P. Linz and S. H. Rodger, *An Introduction to Formal Languages and Automata*, 7th ed. Jones & Bartlett Learning, 2023, Chapter 2. ISBN 9781284231601. Publisher page.